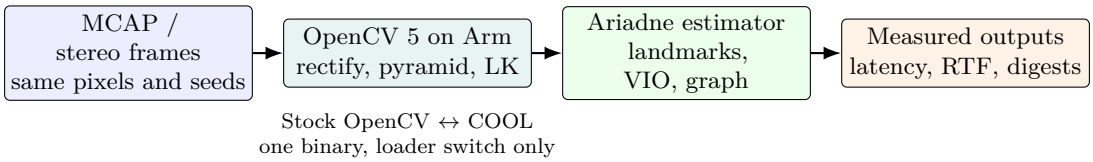
Cloud-Optimized Vision on AWS Graviton4

Ariadne | Best Use of COOL award evidence | Technical companion to the main report

The workload that repeats

Ariadne's stereo agents acquire a new image every 50 ms of simulated time. Each frame passes through camera rectification, photometric preparation, pyramidal Lucas–Kanade (LK) tracking and stereo processing; selected keyframes additionally invoke OpenCV DNN recognition and geometric verification. The cost of these recurring image operations multiplies with the number of agents. Ariadne therefore isolates the OpenCV hot path for native Arm execution, while retaining stereo-inertial estimation, graph optimization, communications and route planning above stable C++ interfaces.

The COOL comparison is anchored to operations that remain useful in the adopted tracker: `cv::remap`, pyramid construction, `cv::calcOpticalFlowPyrLK`, pixel conversion and resizing. Flat-field multiplication, variance-stabilizing transforms and restoration form separately identified radiometric configurations; feature serialization and descriptor indexing are timed at the pipeline boundary without attributing their entire cost to COOL. This workload choice survives the algorithm ablations: XFeat was accurately ported to OpenCV DNN but regressed EuRoC VO on six of seven tested flights, and optional image normalization offered no trajectory gain. ORB/LK continues to supply the high-frequency geometric evidence.



COOL version and AWS configuration

COOL product	AWS Marketplace *Cloud Optimized OpenCV for Graviton4* (`https://aws.amazon.com/marketplace/pp/prodview-fdvbfiewzuehs`): COOL 3.1, an OpenCV 5.x build for Neoverse-V2 with KleidiCV and Arm Performance Libraries, Ubuntu 24.04; C++ SDK under `/opt/cool/cpp_sdk`.
AMI (as published)	`Graviton5-COOL-v3-prod-k6u24vxijyzvg`; eu-north-1 `ami-0a302ff85af8d8764`, published in 18 regions (`infra/cool/cool-regions.tsv`; DECISIONS D28). The AMI name and the listing version label (3.1) are the vendor's.
Instance	c8g.4xlarge (Graviton4, Neoverse-V2, 16 vCPU), eu-north-1, spot; `spinup.sh` profile `cool` (`infra/scripts/spinup.sh`). m8g.4xlarge is the design's concurrent-agent throughput instance (arch §26.3).
Pinned OpenCV	5.0.0; opencv `40738fb16ced`, opencv_contrib `755e50675d97` (`infra/docker/opencv-base.Dockerfile`); x86 build hash `fnv1a64:cf8ea084bb984703`. C++17, Eigen 3.4.1, gcc 13.3, `-O3 -ffp-contract=off`.
Stock Arm HAL	The stock arm64 build already enables carotene + KleidiCV 26.03; dispatch `NEON FP16 *NEON_DOTPROD *NEON_FP16 *NEON_BF16` (`evaluation/runs/arm64/20261023T193621Z-cpuarm-7f8c96/opencv_hal_simd.txt`). A stock-versus-COOL pair is therefore KleidiCV-stock versus COOL.

Execution identity: one binary, loader switch only

Ariadne's build enforces `ARIADNE_REQUIRE_OPENCV=ON` for the full OpenCV-linked target set (root `CMakeLists.txt`). `CMAKE_SKIP_RPATH=ON` and an installed-binary `readelf` gate (`infra/docker/core.Dockerfile`) refuse any embedded RPATH/RUNPATH that would silently defeat the library switch. At execution the manifest records the OpenCV version, `cv::getCPUFeaturesLine()` and an FNV-1a digest of `cv::getBuildInformation()` taken from the *loaded* `libopencv_core`; `/proc/self/maps` names the shared objects actually mapped. The same binary therefore runs on stock OpenCV or on COOL by changing `LD_LIBRARY_PATH` alone, and the manifest proves which one processed the frames.

```
# Build against the COOL SDK on the Graviton4 AMI (or against stock, then switch the loader):
cmake -S . -B build/cool -DARIADNE_REQUIRE_OPENCV=ON \
      -DOpenCV_DIR=/opt/cool/cpp_sdk/lib/cmake/opencv4 -DCMAKE_SKIP_RPATH=ON
```

```
cmake --build build/cool -j
LD_LIBRARY_PATH=/opt/cool/cpp_sdk/lib:$LD_LIBRARY_PATH build/cool/<target>   # COOL
LD_LIBRARY_PATH=/opt/stock/lib:$LD_LIBRARY_PATH        build/cool/<target>   # stock
```

Evaluation method (`infra/cool/day1.sh`, `infra/cool/probe.cpp`)

One command launches the protocol on a fresh COOL instance and writes every step's evidence as it completes:

```
infra/scripts/spinup.sh run --profile cool --type c8g.4xlarge --max-minutes 90 -- 'bash infra/cool/day1.s
```

1. Inspect `/opt/cool`: layout, sonames, CMake package, COOL's own build information and module list.
2. Build stock OpenCV 5.0.0 + contrib natively on the same instance, at the pinned commits with the exact module list and flags of the project image, into `/opt/stock`.
3. Loader-switch probe: one binary built against stock, run with stock and with `/opt/cool/cpp_sdk/lib` first; `/proc/self/maps` gates "same binary, loader switch only".
4. Per-operation output checksums and timings at 1 thread and at all cores, on a 1280×720 frame with the tracker's parameters: `cvtColor`, `remap` (linear), `resize` (area, half size), `GaussianBlur` 5×5, `pyrDown`, `buildOpticalFlowPyramid` and `calcOpticalFlowPyrLK` (window 21, 3 levels), `Sobel`, Otsu `threshold`.

The whole-pipeline stage then replays one recorded stereo MCAP through the same binary on both libraries (the replay harness is the one every Ariadne run uses), reporting p50/p95 per-frame latency, frames per second, CPU utilization, the trajectory digest and pose tolerances. Cost per frame is reported as $C_{\text{frame}} = (C_{\text{instance/h}} + C_{\text{software/h}})/(3600\,\text{FPS})$ with the actual spot and Marketplace prices. Labels stay separate for stock-x86, stock-Arm and COOL-Arm; restoration-only configurations remain auxiliary.

Graviton4 results (stock OpenCV 5.0.0, measured)

Check	Result	Evidence
Native arm64 build + full test suite	OpenCV 5.0.0 + contrib built natively; **708/708** C++ tests pass on the consolidated tree; image gate PASS; commits match the Dockerfile. 901 s, about $0.02.	c8g.4xlarge run `20261024T100530Z-cpuarm-602802`, `evaluation/runs/arm64/20261024T100530Z-cpuarm-602802/`
Cross-architecture determinism	Replay digests identical between x86 and Graviton.	`docs/validation.md` §4e (run `20260929T145114Z-cpu-762cb1`)
Wire bytes across architectures	A FlatBuffers child-construction order difference made arm64 bytes differ; fixed so both architectures emit identical goldens.	commit `6a16717`; arm64 run `05b8cc`
Learned kernels, Arm vs x86	ALIKED 1280×720: **530 ms on c8g vs 829 ms on c7i** (16 threads, ratio 0.64); LightGlue / EigenPlaces ratio 0.85–0.88.	`evaluation/runs/performance/pipeline_profile_20261024/` item 10
Thread invariance	Estimator output identical at OpenCV thread settings 2, 5 and 16; three-agent replay RTF 0.836 on c7i.4xlarge.	main report, §Compute profile

Published COOL figures (vendor, for context)

AWS reports an image-analysis workflow at 1.6 fps on COOL against 1.2 fps with pip-installed OpenCV 4.12 on Graviton4 (about 30%), and up to 70% faster core operations (resize, adaptive Gaussian filtering, contour detection) across 78 functions (https://aws.amazon.com/blogs/spatial/accelerating-opencv-on-graviton-the-cool-framework; https://opencv.org/cool/). Those baselines are pip OpenCV 4.12; Ariadne's stock baseline is OpenCV 5.0.0 with KleidiCV already enabled, so the protocol above measures COOL against the stronger reference.

Evidence entry points

Build and loader controls: `infra/docker/core.Dockerfile`, `infra/docker/opencv-base.Dockerfile`, `docker-bake.hcl`, `infra/scripts/core.sh`, `infra/cool/day1.sh`, `infra/cool/probe.cpp`, `infra/scripts/spinup.sh`, root `CMakeLists.txt`. Workload definition: `docs/architecture.md` §§26.3, 27.1, 38.1 and Iteration 14. Arm runs: `evaluation/runs/arm64/`. x86 integration: `evaluation/runs/gates/premerge_aws/20261024T083919Z-cpux86-73e3fa/` (54/54 checks).

Judge verification: subscribe to the COOL AMI, run the one-command protocol above, and compare the two manifests: the loaded-library list and build-information digest identify stock or COOL, the input digest proves identical frames, and the per-operation and whole-pipeline tables give the speedup at matched outputs.